

Benchmark Test Plan

Memory for coding agents, and why the win is tokens spent better, not fewer of them

The idea, the problem, and what we test

AI coding agents can read a codebase and write code that works, but they keep breaking decisions that were made earlier and don't show up anywhere in the source. Think of a pricing rule the team agreed on three months ago, an authentication constraint, or a convention in the design system. The agent misses these because most memory systems today search by resemblance, and a new task almost never looks like the older decision it actually depends on. So the real problem isn't that the agent spends too many tokens. It's that it spends them on the wrong thing. It confidently writes something that violates a past decision, the work gets rejected in review, and then a few more rounds get burned fixing it. Brief takes a different approach. It stores memory in a structured way, keeping each decision together with the reasoning behind it and linking related decisions to one another, so retrieval can follow the actual dependency instead of guessing. That is what lets the agent get it right the first time. Our paper is the benchmark study that puts this claim to the test. Instead of counting tokens, we measure how much useful work each token buys, looking at how often the agent honors past decisions and what it really costs to reach a correct result. We run this across a range of memory systems, on coding tasks where doing the job well depends on remembering earlier decisions. And before any of that, we run a neutral memory benchmark, so we can show the failure is real on ground we had no part in designing.

What we measure on every benchmark

A few numbers travel with us through every test. The main ones are decision compliance, meaning how often the agent honors a decision made earlier, and whether the final result is actually good enough to merge. Alongside those we track the real payoff, which is the total cost in tokens and dollars to reach a correct result. We also keep some diagnostic numbers, like how often the right memory item was even retrieved and how many tries it took to get the work right. The per call token count we simply report rather than try to shrink, and we hold it roughly steady across systems so the comparison stays fair.

1. Neutral warm up: prove the failure on ground we didn't design

We run these first, exactly as they come. The goal is to show that current memory systems already fail on old or updated decisions, before we bring in any benchmark of our own. This is what protects the paper from the easy criticism that we built a test rigged to make our point.

LongMemEval

Long term conversational memory, using only the temporal and knowledge update questions

WHAT IT TESTS It has 500 questions covering five kinds of memory ability, and we use the two that line up with our claim. The first is temporal reasoning, where the answer depends on something decided a while back. The second is knowledge updates, where a decision has since changed and the old version should no longer apply. The conversation histories run across many sessions and carry timestamps.

HOW WE TEST We run each memory system, from no memory at all to similarity search, graph, and temporal graph, on these two question types. The point is to show that memory based on resemblance misses exactly the cases where the answer depends on an old or replaced fact. There is no coding involved here, which makes it the cleanest possible evidence that the failure is real.

LINK Paper: arxiv.org/abs/2410.10813 Code: github.com/xiaowu0162/LongMemEval Data: huggingface.co/datasets/xiaowu0162/longmemeval-cleaned

2. The coding proof set: where Brief's claim is actually tested

These are the benchmarks that carry the real argument. The question they answer is a simple one. Does structured memory actually help the agent do the right coding work the first time?

dcbench, Brief's own benchmark, and the backbone of ours

Whether a coding agent follows team decisions that are invisible in the code

WHAT IT TESTS It has 8 realistic tasks built around 41 weighted decision points, all on a clean Next.js repo created just for the benchmark. With only the codebase to go on, the baseline agent honors those decisions 46 percent of the time. With Brief added, that jumps to 95 percent, and it does so while using almost the same number of tokens. That last part is the whole point, because it shows the win comes from doing the right work, not from spending less.

HOW WE TEST We build on it rather than just repeat it. We label each task by how many reasoning steps it takes to reach the decision that matters, we add the full range of memory systems instead of only comparing Brief against nothing, and we add two more baselines, one given only a spec and one given only context, so we can tell whether the gain comes from the memory itself or just from having a planning step. Then we run their scoring harness and release our own.

LINK Repo: github.com/brief-hq/dcbench Paper: arxiv.org/abs/2605.08112

DepDec-Bench

Dependency choices that pass the tests but are still the wrong call

WHAT IT TESTS	These are runnable tasks at the repository level, where the agent has to choose dependencies or approaches that carry real security and maintenance consequences, the kind of thing that passing tests will never catch.
HOW WE TEST	We use it to stretch dcbench past a single repo. The lens stays the same. Does the agent make the right engineering call, or does it just produce something that compiles and passes the tests? We run the same set of memory systems on it.
LINK	Paper: arxiv.org/abs/2601.00205

coding-agent-life

Memory across coding sessions, including the causal and temporal cases

WHAT IT TESTS	It contains 15 fictional coding sessions spread across a project that runs about ten days, together with 15 queries that were graded by hand and come with gold session IDs. The queries are already sorted by type, including the multi session causal and temporal ones we care about.
HOW WE TEST	We treat it as a ready made template for trying out our causal and temporal query labels on coding sessions before scaling up. It is small, so we use it to check that the harness and the labeling work, not as a headline result.
LINK	github.com/rohitg00/agentmemory/blob/main/docs/benchmarks/2026-05-20-coding-agent-life-v1.md

SWE-EVO

Long running software evolution on real codebases that touch many files

WHAT IT TESTS	Its tasks come from real release transitions, so the agent has to read release notes and make broad changes that reach across many files in a large repo. The difficulty gap is striking. Top models that score around 73 percent on SWE-bench Verified drop to roughly 23 percent here.
HOW WE TEST	This is our stress test on real code. Tasks that run long and spread across many files are exactly where remembering earlier decisions should pay off, so we measure whether structured memory closes the gap that plain agents fall into.
LINK	Paper: arxiv.org/abs/2512.18470

SWE-bench Verified, used only for contrast

Fixing a single GitHub issue, scored on whether the existing tests pass

WHAT IT TESTS	Each task is one GitHub issue with the whole repo provided, and success is just whether the existing tests pass afterward. Nothing needs to be remembered across sessions, and frontier models already clear it around 80 percent of the time.
HOW WE TEST	We are not using this as a memory test. It is the point of comparison. Agents do well here but fall apart on SWE-EVO and dcbench, and that contrast is our evidence that getting one isolated task right is not the same as remembering decisions across a long project. If we want a tougher reference, SWE-bench Pro is the harder version that is built to resist contamination.
LINK	Site: swebench.com Paper: arxiv.org/abs/2310.06770

How to read this plan. The first section shows the problem is real, using a benchmark we had no hand in building. The second section tests Brief's claim where it actually counts, on coding tasks whose correctness depends on memory, with dcbench as the backbone and the others extending the evidence from one repo out to real code that runs over a long stretch. Every benchmark is run across the same set of memory systems, so the only thing that changes is the memory itself.